

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY

Analysis of algorithms

2

- **Issues:**
 - correctness
 - time efficiency
 - space efficiency
- **Approaches:**
 - theoretical analysis
 - empirical analysis

Empirical analysis of time efficiency

3

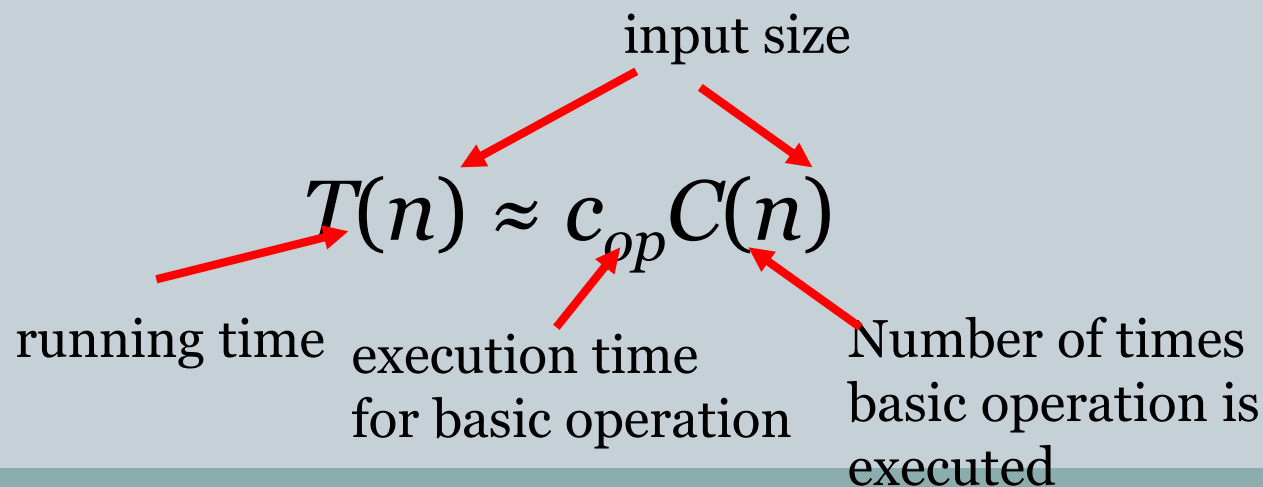
- Select a specific (typical) sample of inputs
- Use physical unit of time (e.g., milliseconds)
or
Count actual number of basic operation's executions
- Analyze the empirical data
- (Vtune , MS profiler)

Theoretical analysis of time efficiency

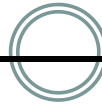
4

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

- Basic operation: the operation that contributes most towards the running time of the algorithm



Input size and basic operation examples



<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers

Best-case, average-case, worst-case

6

For some algorithms efficiency depends on form of input:

- Worst case: $C_{\text{worst}}(n)$ – maximum over inputs of size n
- Best case: $C_{\text{best}}(n)$ – minimum over inputs of size n
- Average case: $C_{\text{avg}}(n)$ – “average” over inputs of size n
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs

Time efficiency of nonrecursive algorithms

7

General Plan for Analysis

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- Determine worst, average, and best cases for input of size n
- Set up a sum for the number of times the basic operation is executed
- Simplify the sum using standard formulas and rules

Example: Sequential search



ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

- Worst case
- Best case
- Average case

Example 2: Maximum element



ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

Example 3: Element uniqueness problem

10

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

Example 4: Matrix multiplication



ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm

//Input: Two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C

Example : Matrix multiplication

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm

//Input: Two n -by- n matrices A and B

//Output: Matrix $C = AB$

for $i \leftarrow 0$ **to** $n - 1$ **do**

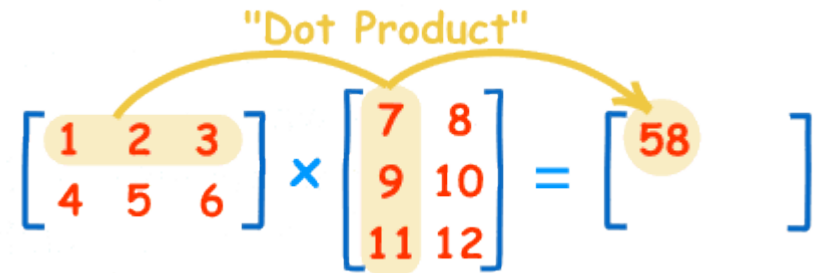
for $j \leftarrow 0$ **to** $n - 1$ **do**

$C[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ **to** $n - 1$ **do** $\sum_{0 \leq i \leq n-1}$

$C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

return C



Input Size: n = matrix order

Basic Operation: Multiplication/Addition

$$M(n) = \sum_{0 \leq i \leq n-1} \sum_{0 \leq j \leq n-1} \sum_{0 \leq k \leq n-1} 1 = n^3$$

Useful Summation Formulas

$$\sum_{i=l}^u 1 = (1 + 1 + \dots + 1) = u - l + 1$$

In Particular: $\sum_{i=1}^n 1 = n - 1 + 1 = n \in \theta(n)$

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2} \in \theta(n^2)$$

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3} \in \theta(n^3)$$

$$\sum_{i=0}^n a^i = a^0 + a^1 + \dots + a^n = \frac{(a^{n+1} - 1)}{(a - 1)} \text{ for any } a \neq 1$$

In Particular: $\sum_{i=0}^n 2^i = 2^0 + 2^1 + \dots + 2^n = 2^{n+1} - 1 \in \theta(2^n)$

Useful Summation Rules

$$\sum_i (a_i \pm b_i) = \sum_i a_i \pm \sum_i b_i$$

$$\sum_i ca_i = c \sum_i a_i$$

$$\sum_{i=l}^u a_i = \sum_{i=l}^m a_i + \sum_{i=m+1}^u a_i$$



Chapter 2

RECURSIVE ALGORITHMS

Chapter Outline

16

- Analyzing recursive algorithms
- Recurrence relations
- Master theorem
- Closest pair algorithms
- Recursion and stacks

Recursive Algorithms

17

- Recursive algorithms solve the problem by solving smaller versions of the problem
 - If the smaller versions are only a little smaller, the algorithm can be called a **reduce and conquer** algorithm
 - If the smaller versions are about half the size of the original, the algorithm can be called a **divide and conquer** algorithm

Recursive Algorithm Analysis

18

- The analysis depends on
 - the preparation work to divide the input
 - the size of the smaller pieces
 - the number of recursive calls
 - the concluding work to combine the results of the recursive calls

Plan for Analysis of Recursive Algorithms

19

- Decide on a parameter indicating an input's size.
- Identify the algorithm's basic operation.
- Check whether the number of times the basic op. is executed may vary on different inputs of the same size. (If it may, the worst, average, and best cases must be investigated separately.)
- Set up a **recurrence relation** with an appropriate initial condition expressing the number of times the basic op. is executed.
- Solve the recurrence (or, at the very least, establish its solution's order of growth) by backward substitutions or another method.

Example 1: Recursive evaluation of $n!$



Definition: $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$ for $n \geq 1$ and $0! = 1$

Recursive definition of $n!$: $F(n) = F(n-1) \cdot n$ for $n \geq 1$ and $F(0) = 1$

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

Size:

Basic operation:

Recurrence relation:

Solving the recurrence for $M(n)$

21

$$M(n) = M(n-1) + 1, \quad M(0) = 0$$

Example 2: Counting #bits



ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

Solving the recurrence for $M(n)$

23

$$M(n) = M(n/2) + 1, \quad M(1) = 1$$

Fibonacci numbers

24

The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

The Fibonacci recurrence:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

General 2nd order linear homogeneous recurrence with constant coefficients:

$$aX(n) + bX(n-1) + cX(n-2) = 0$$

Generic Recursive Algorithm Analysis

25

- The recursive algorithm does the following amount of work:

$$\text{REC}(N) = \begin{cases} \text{DIR}(N) & \text{for } N \leq \text{SizeLimit} \\ \text{DIV}(N) + \sum_{i=1}^{\text{numberSmaller}} \text{REC}(\text{smallerSizes}[i]) + \text{COM}(N) & \text{for } N > \text{SizeLimit} \end{cases}$$

- Where
 - $\text{DIR}(N)$ is the amount of work done by the direct solution
 - $\text{DIV}(N)$ is the amount of work done by the division of the problem
 - $\text{COM}(N)$ is the amount of work done to combine the solutions

Divide and Conquer Example

26

```
Largest( list, start, end )  
if start ≥ end then  
    return list[end]  
end if  
middle = (start + end) / 2  
first = Largest( list, start, middle )  
second = Largest( list, middle+1, end )  
if first > second then  
    return first  
else  
    return second  
end if
```

Analyze this???

Divide and Conquer Example Analysis

27

- **This algorithm does:**
 - One addition and division to calculate middle
 - Two recursive calls with lists about half the original size
 - One comparison to decide between the largest of the first half and the largest of the second half

Divide and Conquer Example Analysis

28

- The number of comparisons of list values done by this example is given by:

$$C(n) = \begin{cases} 0 & \text{for } n = 1 \\ 2C(n/2) + 1 & \text{for } n > 1 \end{cases}$$